

Accenture DevOps — Tier 1 Master Guide

Complete Technical & Interview Preparation: Fundamentals, CI/CD, Git, Docker, Kubernetes & Linux Scripting

1 DevOps Fundamentals

What is DevOps?

DevOps = Development + Operations

DevOps is a combination of culture, practices, processes, and automation that enables development and operations teams to collaborate and deliver software faster and more reliably.

Traditional Model

```
Developer
  ↓
Write Code
  ↓
Throw code to Operations
  ↓
Manual Testing
  ↓
Manual Deployment
  ↓
Production
```

Problems:

- Slow releases
- Communication gaps ("silo mentality")
- Manual errors & fatigue
- Inconsistent environments ("works on my machine")
- Difficult and risky rollback
- Delayed feedback loops

DevOps Model

```
Plan
  ↓
Code
  ↓
Build
  ↓
Test
  ↓
Release
  ↓
Deploy
  ↓
Operate
  ↓
Monitor
  ↓
Feedback
  ↻
```

Key Advantages:

- Continuous delivery & rapid release cycles
- Shared ownership & zero-silo culture
- Automated testing & proactive security
- Repeatable infrastructure as code
- Real-time telemetry and instant observability

ACCENTURE ANGLE & INTERVIEW TIPS

Do NOT answer: "DevOps is a tool."

Instead answer: "DevOps is a culture and set of practices that combines development and operations through collaboration, automation, continuous integration, continuous delivery, monitoring, and feedback."

Accenture's own DevOps job descriptions emphasize **automation, CI/CD, deployment, monitoring, logging, scaling, Linux, and Git.**

CONCEPTUAL TRAP: DEVOPS IS NOT JENKINS

Jenkins is merely an automation server tool. DevOps is an overarching framework comprised of:

```
DevOps
├── Culture
├── Collaboration
├── Automation
├── CI/CD
├── Infrastructure as Code (IaC)
├── Monitoring & Observability
├── Security (DevSecOps)
└── Continuous Feedback
```

DevOps Requirement to Tool Matrix

Requirement / Stage	Industry Standard Tool(s)	Primary Purpose & Role
Version Control	Git, GitHub, GitLab, Bitbucket	Distributed tracking of source code changes & collaboration
CI / CD Engine	Jenkins, GitHub Actions, GitLab CI, Azure DevOps	Automated build, test, and release orchestration
Containerization	Docker, Podman, containerd	Packaging code + dependencies into lightweight portable units
Container Orchestration	Kubernetes (K8s), OpenShift, Amazon EKS	Automated scaling, deployment, networking, and self-healing
Configuration Management	Ansible, Puppet, Chef	Automated configuration drift management & server provisioning
Infrastructure as Code (IaC)	Terraform, AWS CloudFormation, Pulumi	Declarative multi-cloud resource provisioning
Build Tools	Maven, Gradle, npm, yarn	Compiling source, dependency resolution, packaging artifacts
Code Quality & SAST	SonarQube, Snyk, Checkmarx	Static analysis, vulnerability checks, and test coverage gating
Monitoring & Metrics	Prometheus, Datadog, Dynatrace	Time-series metric scraping, threshold alerts, health checks
Visualization / Dashboards	Grafana, Kibana	Real-time visualization of logs, metrics, and application traces
Cloud Platforms		

Requirement / Stage	Industry Standard Tool(s)	Primary Purpose & Role
	AWS, Microsoft Azure, Google Cloud (GCP)	Scalable on-demand compute, storage, and networking

DevOps Lifecycle (Detailed Breakdown)



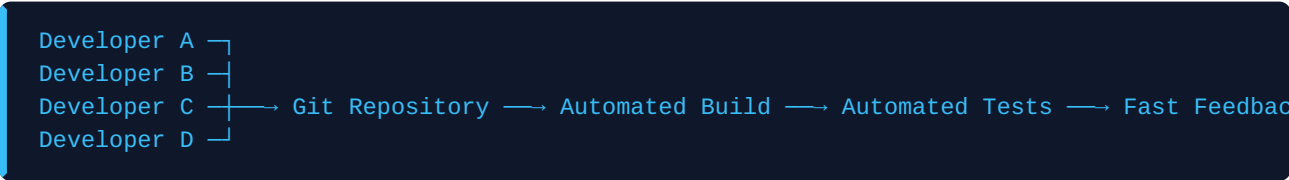
- **PLAN:** Requirements gathering, sprint planning, and issue tracking. *(Tools: Jira, Azure Boards)*
- **CODE:** Developers write and review source code using version control. *(Tools: Git, GitHub, GitLab, Bitbucket)*
- **BUILD:** Compiling code, running linters, and packaging into deployable artifacts (JAR, WAR, Docker images). *(Tools: Maven, Gradle, npm)*
- **TEST:** Automated execution of unit, integration, functional, and security tests. *(Tools: JUnit, Selenium, pytest)*
- **RELEASE:** Tagging, versioning, and publishing verified artifacts to registries (Nexus, Artifactory, ECR).
- **DEPLOY:** Provisioning and deploying artifacts into staging and production clusters. *(Tools: Docker, Kubernetes, AWS, Azure)*
- **OPERATE:** Managing infrastructure runtime, autoscaling, traffic routing, and maintenance.
- **MONITOR:** Continuously collecting metrics, logs, and telemetry for proactive issue resolution:
 - **Metrics tracked:** CPU, Memory, Latency (p95/p99), HTTP Error Rates (5xx), Request Throughput, Logs, and System Availability.

2 CI/CD 🔥🔥🔥

This is one of the highest-priority interview topics. Accenture evaluation panels heavily emphasize pipeline architecture, stages, failure troubleshooting, and delivery vs. deployment distinctions.

CI — Continuous Integration

Developers frequently integrate their code into a shared repository (usually multiple times a day). Automated pipelines build and test the changes immediately upon every push or PR.



Main Objective: Find integration issues, bugs, and breaking changes early before code reaches production environments.

Continuous Delivery

Code is automatically built, tested, packaged, and deployed to staging/pre-prod environments. Every build is **always deployable to production**, but production release is gated by a **manual approval** step.

```
Staging (Passed Tests)
  ↓
Manual Approval
  ↓
Production
```

Continuous Deployment

Every change that passes all stages of the automated pipeline is released directly and **automatically to production without any human intervention**.

```
Git Push → Build → Test
           ↓
Package → Auto-Deploy → Production
```

CORE MEMORIZATION RULE

- **CI:** Integrate code frequently + run automated tests continuously.
- **Continuous Delivery:** Code is always production-ready; production push requires manual gate/approval.
- **Continuous Deployment:** Zero manual gates; passed commits go straight to live production automatically.

End-to-End Enterprise CI/CD Pipeline Flow

```
Developer
  ↓ (git push)
GitHub / GitLab
  ↓ (Webhook trigger)
Jenkins / CI Engine
  ↓
1. Build Application (Maven / npm)
  ↓
2. Run Unit & Integration Tests (JUnit / pytest)
  ↓
3. Static Code Analysis & Quality Gate (SonarQube)
  ↓
4. Docker Image Build & Tagging (docker build -t app:v1)
  ↓
5. Vulnerability Scan (Trivy / Snyk)
  ↓
6. Push Image to Registry (Docker Hub / ECR / ACR)
  ↓
7. Deploy to Kubernetes Staging (kubectl / Helm / ArgoCD)
  ↓
8. Approval Gate / Automated Canary Analysis
  ↓
9. Kubernetes Production Rollout
```

Scenario Question:

Your developer pushes code to the repository. The pipeline automatically triggers, builds the application, and runs automated unit and integration tests. The build passes, and artifacts are deployed to staging. However, deployment to production requires manager/lead approval. What is this workflow called?

Answer: Continuous Delivery.

Explanation: Because there is a manual human approval step gating the production deployment, it qualifies as Continuous Delivery. If the push to production were fully automated upon test pass without any manual sign-off, it would be Continuous Deployment.

Common Accenture Question: How do you troubleshoot a CI/CD pipeline failure?

Answer:

1. **Identify the failed stage:** Check Jenkins/pipeline console logs at the exact failing step (Lint, Unit Test, SonarQube Quality Gate, or Docker Build).
2. **Isolate the error type:**
 - *Compilation/Build errors:* Missing dependency, syntax error, wrong JDK/Node version.
 - *Unit test failures:* Broken assertion or unmocked external dependency.
 - *SonarQube Quality Gate failure:* Code coverage dropped below threshold or new security vulnerability introduced.
 - *Docker build failure:* Base image unavailable, package repository 404, or out-of-disk space.
3. **Local reproduction:** Run the failed step locally using the same environment or container.
4. **Fix and verify:** Push a targeted hotfix commit or notify the author to unblock the master branch.

3 Git & Version Control

Git is an open-source distributed version control system (DVCS) designed to handle everything from small to very large projects with speed and efficiency.

Git Architecture & Internal Stages

```
Developer
↓
Working Directory (Modified files)
↓ git add <file>
Staging Area / Index (Prepared changes)
↓ git commit -m "...
Local Repository (.git directory with commit graph)
↓ git push origin <branch>
Remote Repository (GitHub / GitLab / Bitbucket)
```

Essential Git Commands Reference

Command	Action & Detailed Description
<code>git init</code>	Initializes a new empty Git repository in the current directory.
<code>git status</code>	Displays working directory state, staged/unstaged changes, and untracked files.
<code>git add .</code>	Stages all modified, new, and deleted files in the directory for the next commit.
<code>git commit -m "msg"</code>	Snapshots the staged files into the local repository with a descriptive message.
<code>git push origin <branch></code>	Transfers commits from local repository branch to remote upstream repository.
<code>git pull</code>	Fetches remote updates and merges them into the current branch (<code>git fetch + git merge</code>).
<code>git clone <repo-url></code>	Copies an entire remote repository, history, and branches to local machine.
<code>git branch <name></code>	Creates a new branch pointer at current HEAD.
<code>git checkout <branch></code>	Switches working directory to the specified branch.
<code>git switch <branch></code>	Modern syntax to switch branches safely without conflicting file checkouts.
<code>git switch -c <branch></code>	Creates a new branch and immediately switches to it (<code>git checkout -b</code> equivalent).

Git Branching Workflow

```
main
|
├─ feature-login
├─ feature-payment
└─ bugfix-cart
```

Standard Enterprise Flow:

main → feature branch → development → pull request (PR) → code review & CI checks → merge

Git Merge vs. Git Rebase (Critical Interview Topic)

Git Merge

```
A—B—C (main)
      |
      D—E (feature)
          |
          M (merge commit)
```

- Combines histories and creates an explicit **Merge Commit**.

- **Preserves complete historical context**, original commit timestamps, and branch shape.
- Non-destructive; does not alter existing commit hashes.

Git Rebase

A—B—C—D'—E' (linear main)

- Replays feature commits on top of the target base branch.
- Produces a **clean, linear, and readable git log**.
- **Rewrites commit history** (generates brand new commit hashes for D' and E').

INTERVIEW TRAP & GOLDEN RULE

Never say: *"Rebase is always better than Merge."*

The Golden Rule of Rebase: Never rebase commits that have been pushed to a public/shared repository where other team members are collaborating, as rewriting shared history leads to severe merge conflicts and broken state.

4 Docker & Containers 🔥🔥🔥

Accenture interviews heavily test container mechanics, image optimization, Dockerfile authoring, and container isolation.

What is Docker & The "Works on My Machine" Problem

Docker is an open platform for developing, shipping, and running applications in lightweight, isolated containers. A container packages the **Application Code + Runtime + System Libraries + Dependencies + Configurations** into an immutable unit.

The Classic Mismatch Problem

Developer Machine:

- Node.js v22
- MongoDB v7.0
- Library A v5.2

Production Server:

- Node.js v18 (Mismatch!)
- MongoDB v4.4 (Mismatch!)
- Library A v3.0 (Crash!)

The Docker Solution

```
Application
+
Runtime & Dependencies
+
Libraries & Config
↓
Docker Image (Immutable Blueprint)
↓
Docker Container (Identical everywhere)
```

Eliminates environmental drift between Dev, QA, Staging, and Production.

Docker Image vs. Docker Container

Docker Image

- A **read-only, immutable template** containing layered instructions and filesystem.
- Acts as the **blueprint**.
- Examples: `node:22`, `nginx:latest`, `ubuntu:24.04`

Docker Container

- A **running, active instance** of a Docker image.
- Adds a thin, writable container layer on top of the image layers.
- Created via `docker run`.

Docker Architecture & Core Commands

Docker Client (CLI) —REST API—> Docker Daemon (dockerd) —> Pulls Images / Runs Containers

Command	Purpose & Description
<code>docker pull nginx</code>	Downloads the nginx image from remote registry (Docker Hub) to local storage.
<code>docker images</code>	Lists all locally stored Docker images with tags and sizes.
<code>docker ps</code>	Lists currently active/running containers.
<code>docker ps -a</code>	Lists all containers (both running and stopped/exited).
<code>docker run -d -p 80:80 nginx</code>	Creates and starts a container in detached mode with port forwarding.
<code>docker stop <container_id></code>	Gracefully stops a running container (sends SIGTERM, then SIGKILL).
<code>docker rm <container_id></code>	Removes a stopped container from the local host.
<code>docker rmi <image_id></code>	Deletes an image from local storage.

Dockerfile Breakdown & Build Flow

```
# 1. Base Image
FROM node:22

# 2. Set Working Directory inside container
WORKDIR /app

# 3. Copy package manifests first for Docker layer caching
COPY package*.json ./

# 4. Install dependencies
RUN npm install

# 5. Copy remaining source code
COPY . .

# 6. Expose the runtime port
EXPOSE 3000

# 7. Default execution command
CMD ["npm", "start"]
```



Docker CMD vs. ENTRYPOINT

Directive	Primary Role	Override Behavior
ENTRYPOINT	Defines the fixed executable binary that runs when container launches.	Cannot be overridden easily (requires explicit <code>--entrypoint</code> flag).
CMD	Provides default arguments or a fallback command for the container.	Easily overridden by passing CLI arguments at runtime (<code>docker run app echo "hi"</code>).

Multi-Stage Docker Builds 🔥

In standard Docker builds, build tools (compilers, SDKs, devDependencies) inflate image size and expose security risks. Multi-stage builds use multiple `FROM` statements to isolate the build environment from the lean final runtime image.

```
# Stage 1: Build Phase (Includes Node.js SDK & devDependencies)
FROM node:22 AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Stage 2: Lean Production Runtime (Only Nginx + compiled HTML/JS static bundle)
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

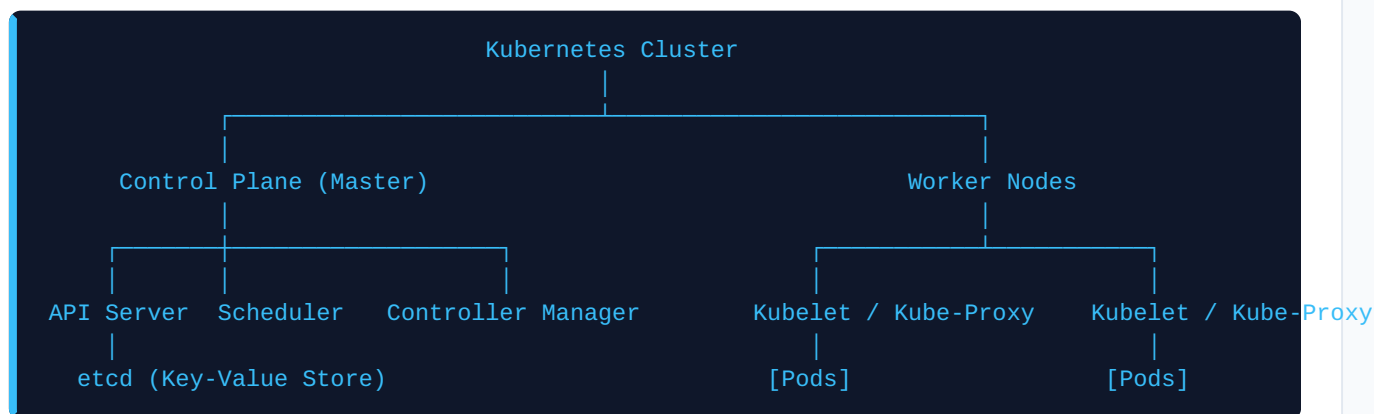
Key Multi-Stage Benefits:

- **Dramatically smaller final image size:** Drops an image from ~1GB to ~25MB.
- **Reduced attack surface:** Compilers, git, and dev packages are completely excluded from production.
- **Zero bloat:** Only compiled binaries and static artifacts ship to production servers.

5 Kubernetes Basics 🔥🔥🔥

Docker runs individual containers on a single host. But in enterprise environments with 500+ containers across 50+ virtual servers, manual management is impossible. **Kubernetes (K8s)** is an open-source container orchestration platform that automates deployment, scaling, healing, and network routing.

Kubernetes Architecture



Core Kubernetes Components

- **Pod 🔥:** The smallest deployable unit in Kubernetes. Represents a single instance of a running process.
 - Usually runs 1 main application container.
 - Can host multiple tightly-coupled containers (e.g., App container + Sidecar logging proxy container) sharing the same network namespace (**localhost**) and storage volumes.
- **Node:** A worker machine (VM or physical server) that runs Pods, managed by the Control Plane.
- **Deployment:** A declarative controller that manages replicated Pods, handling automated rollouts, health checks, self-healing, and zero-downtime rolling updates.

```
replicas: 3  —→  [Pod 1] [Pod 2] [Pod 3]
If [Pod 1] crashes  —→  K8s Controller immediately spawns [Pod 4] to maintain 3 healthy replicas.
```

- **Service:** Pods are ephemeral with dynamic, changing IP addresses. A Kubernetes Service provides a stable, persistent virtual IP and DNS name to balance traffic across matching Pods.

ConfigMap vs. Secret

Feature	ConfigMap	Secret
Data Stored	Non-sensitive configuration data	Sensitive, confidential credentials
Typical Examples	<code>DATABASE_HOST</code> , <code>API_URL</code> , <code>APP_MODE</code>	Database Passwords, API Keys, TLS Private Keys, Tokens
Storage Format	Plain UTF-8 text strings	Base64 encoded (can be encrypted at rest via KMS in etcd)

Kubernetes NetworkPolicy

Scenario Question:

In a microservices application, Frontend Pods must communicate with Backend Pods, but the Database Pod should strictly reject all incoming traffic originating directly from the Frontend Pods. How do you enforce this restriction in Kubernetes?

Answer: By configuring a Kubernetes NetworkPolicy.

Explanation: By default, pod networking is non-isolated (any pod can talk to any pod). A

`NetworkPolicy` resource uses pod selectors and label matchers to define ingress/egress rules, restricting traffic to the Database pod so that it only accepts connections from Pods carrying the label `app: backend`.

6 Linux & Shell Scripting

Linux is the foundation of modern cloud and DevOps environments. Engineers must be proficient in managing processes, modifying permissions, inspecting system logs, and writing bash scripts.

Essential Linux Commands Cheat Sheet

Category	Command	Usage & Meaning
File & Directory	<code>pwd</code>	Print current working directory path.
	<code>ls -la</code>	List all directory contents including hidden files with long permissions.
	<code>cd /path</code>	Change directory to the specified destination.

Category	Command	Usage & Meaning
	<code>mkdir -p dir/sub</code>	Create directories recursively without throwing errors if existing.
	<code>touch file.txt</code>	Create an empty file or update existing file timestamps.
	<code>cp -r src dest</code>	Copy files or directories recursively.
	<code>mv / rm -rf</code>	Move/rename files, or forcefully remove directories recursively.
Processes & System	<code>ps aux</code>	Display snapshot of all currently active system processes.
	<code>top / htop</code>	Real-time interactive monitoring of CPU, RAM, and process consumption.
	<code>kill -9 <PID></code>	Send forceful termination signal (SIGKILL) to process ID.
Logs & Searching	<code>tail -f app.log</code>	Continuously stream live log additions to console in real-time.
	<code>grep -i "ERROR" log</code>	Search and filter log lines containing case-insensitive string "ERROR".

Linux Permissions & Octal Calculation

Linux file permissions are represented as 9 characters grouped into three triads:

```

Permission String: -rwxr-xr--
Triad Breakdown:  [Owner: rwx]  [Group: r-x]  [Others: r--]
Numerical Values:  4 + 2 + 1    4 + 0 + 1    4 + 0 + 0
Octal Notation:    7            5            4

```

Octal Bit Weightings: Read (r) = 4 , Write (w) = 2 , Execute (x) = 1

- `chmod 755 script.sh` gives Owner full read/write/execute (7), and Group/Others read/execute permissions (5).

Shell Scripting — Factorial Program

Recent Accenture DevOps coding rounds specifically ask candidates to write clean bash shell scripts (e.g., calculating factorial, checking service health, parsing logs).

```
#!/bin/bash

# Prompt user for integer input
read -p "Enter number: " n

# Validate non-negative integer
if [[ $n -lt 0 ]]; then
    echo "Factorial is not defined for negative numbers."
    exit 1
fi

fact=1

# Loop from 1 to n to compute factorial
for ((i=1; i<=n; i++))
do
    fact=$((fact * i))
done

echo "The factorial of $n is: $fact"
```